

A Remotely Accessible GridPACK–NS-3–HELICS Testbed for Synchrophasor-Based Monitoring and Control Under Cyber Attacks

Md Mazharul Islam, Sunjare Zulfiker, Masum Reza, and Hafiz Abdur Rahman

Abstract—Phasor measurement units (PMUs) give control centers time-stamped phasors many times per second, but their value depends on the network that carries the frames and on whether the frames can be trusted. Hardware testbeds for this chain are costly and serve few users, while open-source co-simulations target distribution systems or omit the measurement chain and its defenses. This paper presents an open-source, remotely accessible testbed for the complete synchrophasor loop under network impairments and cyber attacks, and measures which parts of the loop decide whether an attack succeeds. GridPACK solves the power flow as a persistent message passing interface (MPI) job, NS-3 carries every PMU frame and command through a phasor data concentrator (PDC), and a control center performs state estimation, chi-square bad data detection with normalized-residual removal and voltage control. HELICS synchronizes the federates, and users work from a web dashboard. On the IEEE 118-bus system, 225 runs combined an automatic voltage regulator fault with four attacks, two PMU placements, five link latencies and five PDC wait windows, each with ten random seeds. With 32 PMUs, detection without removal, delay and drop each stretched the over-voltage from 0.5 to 5.5 s in every seed, and with 68 PMUs no attack extended it. Without any attack, 40 ms of mean link latency and a 20 ms wait window hid the fault in six of ten seeds. Removing bad data, redundant placement and a wait window chosen from the delay spread therefore matter as much as the detection test.

Index Terms—Co-simulation, cyber-physical systems, false data injection, GridPACK, HELICS, NS-3, phasor data concentrator, phasor measurement unit, state estimation, testbed.

I. INTRODUCTION

FOR decades, transmission control centers have watched the grid through supervisory control and data acquisition (SCADA). Remote terminal units report unsynchronized magnitudes every few seconds, and the state estimator needs iterations over a nonlinear model to turn them into bus voltages [1]. A phasor measurement unit (PMU) changes this picture. It samples the voltage at its bus and the currents on the branches connected to it, stamps each frame with time from the global positioning system (GPS), and sends 30 to 60 frames per second to a phasor data concentrator (PDC) [2], [3]. The PDC aligns the frames by time stamp and passes sets to applications in the control center. Because the phasors are linear in the bus

voltages, the control center can estimate the state in one step and act on it within a fraction of a second [1], [2].

Two things limit what the control center can do with these data. The first is the communication network. A frame that arrives after the PDC has released its time stamp is lost for that set, and a set that misses PMUs can leave part of the grid unobserved [4], [5]. The second is trust. An attacker who can change frames can try to hide a real problem or create a false one. Liu *et al.* showed that an attacker who knows the measurement model can add an error that the residual test of the estimator cannot see [6], and later studies examined how many measurements such an attack needs, how to protect them and how to catch the attack with learned detectors [7]–[10]. Attacks on timing are just as effective. Shifting the time reference of a PMU or delaying its frames makes the PDC drop them [11], [12].

These questions cross the boundary between the power system and the network, so studying them requires both to be simulated together, with the PMUs, the PDC and the control center in between. Real-time hardware testbeds do this with high fidelity [13]–[16], but they cost a great deal, and the number of experiments they can run at once is fixed by the hardware. Open-source co-simulation platforms built on HELICS [17], [18] lower the cost, yet the recent ones model distribution feeders, advanced metering or distributed energy resources [19]–[22], or couple transmission and distribution without any network [23]. Our own earlier prototype coupled GridPACK and NS-3 on the IEEE 14-bus system and studied distributed denial of service (DDoS), but it had no PMU chain, no estimator and no closed loop [24]. None of these platforms lets several users run transmission-level synchrophasor experiments from a browser on shared computing resources, and none reports, for one system and one fault, how the PDC wait window, the spread of link delays, the redundancy of PMU placement and the choice between detecting and removing bad data change the outcome of an attack.

This paper fills that gap with an open-source testbed and an experimental study on the IEEE 118-bus system. Its contributions are as follows.

- 1) A co-simulation of the complete synchrophasor loop in which GridPACK [25] runs as a persistent message passing interface (MPI) job, NS-3 [26] carries every PMU frame and every control command through a PDC with a wait window, and a control center estimates the state, tests and cleans it, and corrects voltage violations, all synchronized by HELICS. We derive the closed-loop

The authors are with the Department of Electrical and Computer Engineering, North South University, Dhaka 1229, Bangladesh (e-mail: mazharul.islam1@northsouth.edu; sunjare.zulfiker@northsouth.edu; masum.reza@northsouth.edu; hafiz.rahman@northsouth.edu).

Corresponding author: Hafiz Abdur Rahman (e-mail: hafiz.rahman@northsouth.edu).

This work was supported by the Bangladesh Energy and Power Research Council (EPRC) under grant EPRC/58-2018-007-01.

latency that the time coupling creates and confirm it in the logs.

- 2) An operating layer for remote, multi-user use on a cluster: a queue that starts an experiment only when its MPI ranks fit in the free slots, one HELICS broker and port per experiment, placement of federates on nodes, scenario files, an observer federate that streams each run to a Node-RED dashboard [27], and plain-file outputs for every run.
- 3) A study of 225 runs on the IEEE 118-bus system: 21 scenarios that combine an automatic voltage regulator (AVR) fault with simple and stealthy false data injection (FDI), delay and drop attacks, two PMU placements, five link latencies and five PDC wait windows, each with ten random seeds, plus a sweep of MPI ranks. Baselines, ablations and statistical tests separate the effect of each defense.
- 4) Three findings that follow from the study: removing the suspect PMU matters as much as detecting the attack, redundant placement defends against attacks that withhold measurements, and the PDC wait window must follow the spread of link delays. The code, scenario files, scripts and run summaries are public so that every number in this paper can be regenerated.

The rest of this paper is organized as follows. Section II gives the background on synchrophasors, PMU state estimation, attacks and co-simulation. Section III reviews related testbeds and studies by theme and compares them in Table I. Section IV presents the proposed testbed and its algorithms. Section V describes the implementation, the experimental setup, the baselines and the metrics. Section VI reports the results, including ablations and statistical tests. Section VII discusses the findings, the limitations and the threats to validity, and Section VIII concludes the paper.

II. BACKGROUND

A. Synchrophasors, PMUs and PDCs

A synchrophasor is the phasor of a voltage or current referred to a common time reference. IEEE C37.118.1 defines its measurement and accuracy, and C37.118.2 the frames that carry it [3], [28]. A PMU reports a frame at a fixed rate $1/T$, for example $T = 1/30$ s. Frames from many PMUs reach a PDC, which buffers them by time stamp. Because a PDC cannot wait forever for a late frame, it releases a set when all expected PMUs have reported or when a wait time has passed [4]. The wait time trades latency for completeness, and in hierarchical networks the wait times of several PDCs have to be set together [5]. Measured end-to-end latencies of wide-area systems are in the tens of milliseconds [29], [30].

B. PMU State Estimation and Bad Data

If every PMU channel is a linear function of the complex bus voltages $V \in \mathbb{C}^n$, the m received channels satisfy $z = HV + e$, where e is the measurement error. The weighted least-squares (WLS) estimate minimizes the weighted squared residual and has a closed form [1]. Under Gaussian errors, the weighted sum of squared residuals J follows a chi-square

distribution with $2m - 2n$ degrees of freedom when the data are good, so J above a quantile of that distribution signals bad data. The largest normalized residual then points to the measurement most likely to be wrong [1], [31]. A grid is observable from PMUs when every bus has a PMU or is adjacent to one, and the minimum set of PMUs that achieves this is found by integer programming [32], [33].

C. Attacks on Synchrophasor Data

An attacker who adds $a = Hc$ to the measurements, for any c , shifts the estimate by c and leaves the residual unchanged, so the chi-square test cannot see it [6]. Such a stealthy attack needs every measurement that depends on the shifted states. Kosut *et al.* and Bobba *et al.* studied how many measurements an attacker needs and which ones a defender should protect [7], [8]. Recent work builds AC attacks that pass the test with high probability [34] and detectors that learn the normal data to catch them [9], [10]. Attacks can also withhold data. Time-synchronization attacks and added delay make frames miss the PDC [11], [12], [35], and flooding a PMU's connection can close it [36]. Placement can help. If the grid stays observable after the attacked PMUs are removed, the attack can be located and corrected [37], and placement can be chosen to make attacks harder [38].

D. Co-Simulation of Power and Communication Systems

A co-simulation runs separate simulators for the grid and the network and exchanges data and time between them. Early platforms such as EPOCHS synchronized at fixed points [39], GECO used a global event queue [40], and surveys compare the coupling methods [41]. Mosaik [42] and HELICS [17], [18] are general frameworks. In HELICS each simulator is a federate that publishes values and requests the next time it needs. A broker grants a time only when no other federate can still send an earlier message, which keeps causality across federates written in different languages [18]. Real-time co-simulation adds the constraint of wall-clock pacing and its own interfacing problems [43], [44].

III. RELATED WORK

A. Hardware and Real-Time Testbeds

Hahn *et al.* set out the architecture and evaluation of cyber-physical security testbeds [45], and PowerCyber opened such a testbed with real-time simulators and field devices to remote users [46]. NEFTSec federated testbeds at several sites to study wide-area protection [47]. Recent real-time platforms couple OPAL-RT or RTDS simulators with network emulators. Kumar *et al.* protected wide-area damping control against FDI and lost signals with a learned signal reconstruction [13]. Nguyen *et al.* automated many attack scenarios on a large-scale testbed [14]. Das *et al.* rebuilt IEEE C37.118.2 frames to inject false data against bad data detection [15]. Mishchenko *et al.* demonstrated attacks on synchrophasor streams in a multidomain testbed and left the defender side to future work [16]. Others built hybrid physical laboratories [48], training testbeds [49], hardware-in-the-loop platforms with commercial

network simulators [50], and a cloud cyber range [51]. These systems give high fidelity, but each depends on licensed hardware at one site, and the number of experiments that can run at once is fixed by that hardware.

B. Open-Source Co-Simulation Platforms

HELICS has become the common choice for open-source co-simulation [18]. NATIG packages GridLAB-D, NS-3 and HELICS in containers and generates attacks on inverter setpoints in distribution microgrids [19]. GridSecSim couples GridLAB-D and NS-3 for advanced metering and trains an intrusion detector [20]. Vallam Kondu *et al.* attach an NS-3 module to a distributed energy resource management system and measure the communication impact of attacks [21]. Masood *et al.* show that a denial of service can mask the effect of a false data injection on a distribution grid [22]. Zheng *et al.* keep GridPACK's MPI parallelism inside a HELICS federation to co-simulate transmission and distribution faster than real time, but without a network [23]. Our earlier prototype added a Node-RED front end to GridPACK and NS-3 on the IEEE 14-bus system [24]. None of these platforms models a PMU, a PDC and a transmission state estimator in the loop.

C. Synchrophasor Networks, Latency and the PDC

Several studies treat the network that carries synchrophasors. Castello *et al.* built a validated platform for measuring the latency of a transmission operator's wide-area system [29]. Morato *et al.* simulated PMU traffic over 5G and found mean latencies near 20 ms [30]. Pourramezan *et al.* optimized the wait times of hierarchical PDCs for throughput and observability [5]. Jha *et al.* modelled the resiliency of synchrophasor networks as a graph and checked it in ns-3 [52]. Zadsar *et al.* showed that the PDC wait time is itself a defense against time-synchronization attacks [11]. These works study the network or the PDC alone and do not close the loop through an estimator and a controller.

D. Attacks and Defenses on PMU State Estimation

Detection studies usually work on recorded or generated data sets. Chen *et al.* and Bhattacharjee *et al.* detect stealthy FDI with unsupervised and self-supervised models on IEEE test systems including the 118-bus system [9], [10]. Yang *et al.* built AC attacks that passed a chi-square test in 99.3% of cases [34]. Khalafi *et al.* placed PMUs so that the grid stays observable without any one PMU and located attacks by repeated estimation [37]. Pourahmad *et al.* placed PMUs and PDCs to reduce attack possibilities and communication cost on the IEEE 30- and 118-bus systems [38]. Shereen and Dán bounded the time references an undetectable time-synchronization attack must control [12]. These methods are evaluated offline, without a simulated network or a controller that acts on the estimate.

E. Comparison and Research Gap

Table I summarizes representative works by method, test system, metrics, main result and limitation. Three gaps stand

out. Testbeds that model the synchrophasor chain rely on real-time hardware at one site. Open-source platforms with a network simulator model distribution systems and have no PMU estimator in the loop. And none of the studies measures, on one transmission system, how the PDC wait window, the spread of link delays, the redundancy of placement and the treatment of detected bad data together decide whether an attack hides a fault from the controller. The testbed and the study in this paper address these three gaps.

IV. PROPOSED TESTBED

A. Architecture

Fig. 1 shows the architecture of one experiment. Three federates form the closed loop. The grid federate drives GridPACK and publishes the true phasors that each PMU would measure. The network federate, an NS-3 program, turns them into PMU frames, carries them over a link per PMU to a PDC, and releases time-aligned sets. The control-center federate estimates the state, tests it, and sends generator setpoint commands, which return through NS-3 to the grid. A fourth federate, the observer, only subscribes and copies the state of the run to the dashboard, so no federate in the loop waits for it. Table II lists the HELICS publications. Around the federation, a FastAPI service [53] queues experiments, starts the broker and the federates, and serves results to the Node-RED dashboard used by remote users. Fig. 7 shows how these parts are placed on the cluster used in this paper.

B. Grid Federate and the GridPACK Server

GridPACK distributes a network over MPI processes, partitions it with ParMETIS and solves the network equations with PETSc [25], [54]. Its applications normally run once per study. A co-simulation needs a new solution at every grid step, and restarting an MPI job each step would spend most of the time on start-up and partitioning. We therefore wrote a small server, `pf_server`, around GridPACK's power flow. It is started once per experiment with `mpirun`, reads the case and partitions the network once, and then waits for commands on the standard input of rank 0 (Table III). Rank 0 broadcasts each batch, each rank applies the changes to the buses it owns, GridPACK solves the power flow by Newton–Raphson with the MUMPS direct solver [55], and the bus voltages are gathered to rank 0 [56]. Replies start with @@ so that the driver can separate them from the solver's own output, which it passes to the dashboard. Zheng *et al.* kept GridPACK's parallelism under HELICS in a similar way by making rank 0 the federate [23]. Our server keeps HELICS out of the MPI program, so the same driver can fall back to a built-in solver if the MPI job fails.

The grid federate is a Python program that drives the server. At every grid step Δ it applies the commands that have arrived over the network, updates the loads, solves, publishes the phasors, and logs the true voltage of every bus so that the experiment can be scored afterwards. Each load follows a slow sinusoid plus an independent random walk, which keeps the measurements from being constant. Events, such as the AVR fault used in this paper, are applied at their scheduled grid step.

TABLE I
COMPARISON OF RELATED TESTBEDS AND STUDIES

Work	Method or platform	Test system	Metrics	Main result	Limitation
[13]	OPAL-RT hardware in the loop, CNN-LSTM reconstruction of PMU feedback	105-bus Indian grid	Damping of inter-area modes	Damping kept under FDI and lost signals	Licensed hardware, network not simulated
[14]	OPAL-RT and RTDS with EXata emulation, scripted scenarios	IEEE test feeders	Impact on operation	Many attack scenarios automated	Licensed tools, no synchrophasor estimation
[15]	Real-time simulator, C37.118.2 frame rewriting, virtual machines	IEEE 13-node feeder	Bad data detection	FDI on PMU data shown against detection	Hardware, no closed loop, PDC timing not studied
[16]	OPAL-RT with virtual PMUs and adversary setup	Aggregated dynamic model	Attack feasibility	Attacks on C37.118.2 streams demonstrated	Defender side left to future work
[19]	GridLAB-D, NS-3 and HELICS in containers	IEEE 123-bus, 9500-node	Voltage and frequency deviation	Inverter types react differently to set-point attacks	Distribution only, no PMUs or estimator
[20]	GridLAB-D, NS-3 and HELICS, intrusion detector on a single-board computer	IEEE 13-node, ten meters	Precision, recall, F1	F1 of 99.5% on metering traffic	Metering only, no grid control
[23]	GridPACK on MPI with GridLAB-D under HELICS	3000-bus with 100 feeders	Run time, accuracy	30 s simulated in 24.4 s on 4 processes	No network or attacks
[24]	GridPACK, NS-3, HELICS, Node-RED	IEEE 14-bus	Latency, loss, response time	Latency rose from 10 to about 100 ms under DDoS	DDoS only, no PMU chain or control
[5]	Model of hierarchical PDC wait-time policies	Realistic PMU data	Throughput, observability	Optimal wait times found	No attacks, no closed loop
[11]	Co-optimized PMU-PDC tree and PDC wait time	Kundur, IEEE 39-bus	Modal observability	Wait time limits data dropout under attack	Model-based, network not simulated
[37]	N-1 placement, repeated estimation, clustering	Sample system	Detection, localization	Stealth impossible if observable after removal	Static, no network
[9]	Unsupervised detector of stealthy FDI	IEEE 39- and 118-bus	Detection performance	Catches attacks missed by residual tests	Offline data, no network
[34]	AC FDI by tensor principal component analysis	IEEE 5- to 3120-bus	Attack success rate	99.3% success against chi-square test	Attack side only
This work	GridPACK on MPI, NS-3 PMUs and PDC, WLS with chi-square test and removal, voltage control, HELICS, remote dashboard	IEEE 118-bus	Violation time, complete and observable sets, alarms, latency, run time	Removal, placement and PDC wait decide whether an attack hides a fault	Quasi-steady state, JSON frames, virtual cluster

TABLE II
HELICS PUBLICATIONS

Topic	From, to	Content
grid/meas	grid, NS-3	True phasors of every PMU channel, each grid step
ns3/pdc	NS-3, control center	Time-aligned PMU sets released by the PDC
cc/commands	control center, NS-3	Generator setpoint commands
ns3/cmd_delivered	NS-3, grid	Commands that reached the generator node, with arrival time
grid/status	grid, observer	True bus voltages, commands applied, solver report
ns3/status	NS-3, observer	Per-PMU frame counters, PDC and command counters
cc/status	control center, observer	Test result, removed PMUs, estimate, computing time

C. Network Federate: PMUs, Links and PDC

Every PMU is an NS-3 node with its own point-to-point link to the control-center node. The one-way delay of link i is drawn once per experiment as

$$d_i = L u_i, \quad u_i \sim \mathcal{U}(0.5, 1.5), \quad (1)$$

TABLE III
PROTOCOL OF PF_SERVER, ONE EXCHANGE PER GRID STEP

Direction	Line	Meaning
to rank 0	LOAD bus P Q	new load at a bus
to rank 0	VSET bus id V	new generator setpoint
to rank 0	SOLVE / QUIT	solve the batch / stop
from rank 0	@@RANK rank host buses branches	partition of each rank
from rank 0	@@READY nbus nrank	network read, partitioned
from rank 0	t_setup	
from rank 0	@@RESULT ok nbus t_apply	solve finished, timings
from rank 0	t_solve	
from rank 0	@@V bus Vm Va ... @@END	voltage of every bus

where L is the mean latency. Each frame also waits 2 ms of processing plus a jitter drawn from $\mathcal{U}(0, 2)$ ms, and a packet error model loses frames with a fixed probability. Each generator has its own node and link, which carry commands.

At every frame time kT , each PMU takes the latest phasors published by the grid federate and adds noise. With s the true phasor and $\epsilon_1, \epsilon_2 \sim \mathcal{N}(0, 1)$, the reported phasor is

$$p = |s| (1 + 0.001 \epsilon_1) e^{j(\angle s + 0.001 \epsilon_2)}. \quad (2)$$

The PMU sends the frame as a user datagram protocol

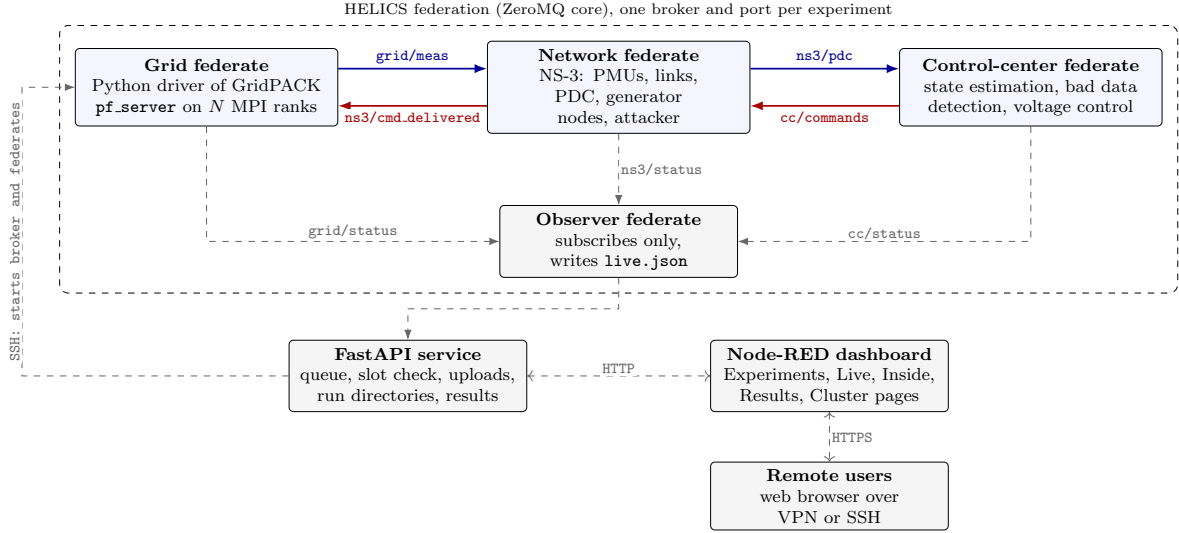


Fig. 1. Architecture of the testbed for one experiment. Solid blue arrows carry measurements, solid red arrows carry commands, and dashed arrows monitor and control the experiment.

(UDP) packet carrying its number, time stamp and phasors in JavaScript Object Notation (JSON). The PDC keeps a buffer per time stamp and releases a set as soon as every PMU has reported, or a wait window w after the first frame of that time stamp arrived [4]. A frame that arrives after its set was released is counted as late. Algorithm 1 gives one frame interval, including the attacks of Section IV-F.

D. PMU Placement

A PMU at bus i measures the voltage of i and the current of every in-service branch at i , so it makes i and its neighbours observable. With A the bus connectivity matrix including the diagonal and $x \in \{0, 1\}^n$ the placement, the testbed solves

$$\min_x \mathbf{1}^\top x \quad \text{s.t.} \quad Ax \geq \kappa, \quad (3)$$

as in [33], where $\kappa_i = \min(k, \deg(i) + 1)$. Here $k = 1$ gives the minimum placement for full observability and $k = 2$ a redundant placement in which every bus is seen twice where its degree allows. The problem is solved with the HiGHS solver [57]. For the IEEE 118-bus system the minimum placement has 32 PMUs with 169 phasor channels (Fig. 2) and the redundant placement 68 PMUs with 315 channels (Fig. 3). Bus 49, where the fault and the attacks of Section V take place, has a PMU in both. Under the minimum placement it is observed by PMUs 45 and 49, and under the redundant placement by PMUs 42, 45, 49, 51 and 66.

E. Control Center

State estimation. With all quantities complex, the channels of the received PMUs satisfy $z = HV + e$, where a row of H is a unit vector for a voltage channel and a row of the branch admittance matrices for a current channel. The WLS estimate is

$$\hat{V} = G^{-1}H^H W z, \quad G = H^H W H, \quad (4)$$

Algorithm 1 Network federate, frame interval k

Input: frame period T , link delays d_i , wait window w , attack settings

- 1: request HELICS time kT
- 2: publish the PDC sets and command deliveries of the last interval
- 3: read new grid phasors s_i and commands; send each command to its generator node
- 4: $a \leftarrow$ attack active at kT
- 5: **if** a is an FDI and the attacker holds the PMU at target bus b **then**
- 6: $c \leftarrow (v_f - |s_{b,0}|) e^{j\angle s_{b,0}}$ ▷ adaptive offset
- 7: **end if**
- 8: **for** each PMU i **do**
- 9: $p \leftarrow s_i$ with noise by (2); $e \leftarrow 0$
- 10: **if** a and PMU i is attacked **then**
- 11: **if** drop **then** mark the frame dropped; **continue**
- 12: **end if**
- 13: **if** FDI **then** add $h_{kb} c$ to attacked channels
- 14: **end if**
- 15: **if** delay **then** $e \leftarrow d_a$
- 16: **end if**
- 17: **end if**
- 18: send p after $2 \text{ ms} + \mathcal{U}(0, 2) \text{ ms} + e$; the link adds d_i
- 19: **end for**
- 20: run NS-3 events until $(k+1)T$; for each frame of time stamp k' that reaches the PDC: if set k' was released, mark it late; else store it, release set k' if complete, and if it is the first frame of k' , schedule the release at arrival $+w$

with $W = \text{diag}(1/\sigma_k^2)$ and $\sigma_k = \max(\sigma|z_k|, 10^{-4})$. G is Hermitian positive definite and is factorized once per set with a sparse LU in symmetric mode. A set is observable when every bus is the own bus or a neighbour of a received PMU.

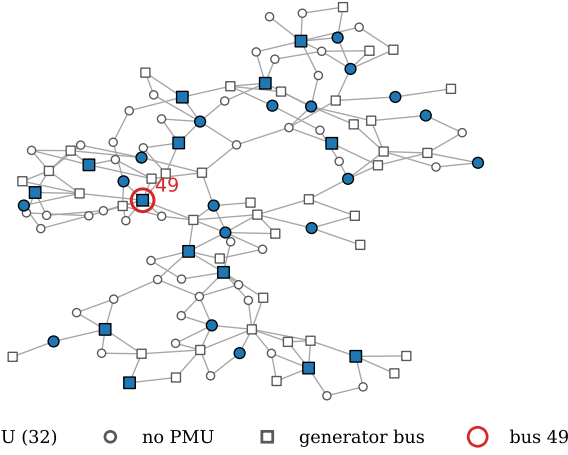


Fig. 2. Minimum PMU placement for the IEEE 118-bus system computed by (3) with $k = 1$: 32 PMUs.

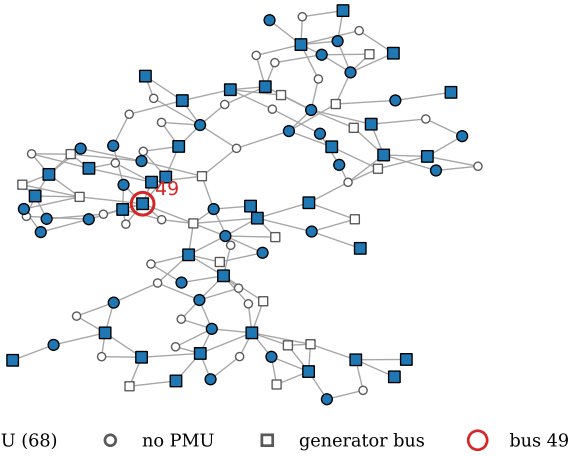


Fig. 3. Redundant PMU placement for the IEEE 118-bus system computed by (3) with $k = 2$: 68 PMUs.

When it is not, the estimator adds weak pseudo-measurements equal to the previous estimate with standard deviation 0.05 pu, so that it still returns a full state, and the set is not tested.

Bad data detection and removal. For an observable set, the statistic

$$J = \sum_{k=1}^m \frac{|z_k - h_k \hat{V}|^2}{\sigma_k^2} \quad (5)$$

is compared with $\tau = \chi_{2m-2n}^2(1 - \alpha)$ [1]. For the IEEE 118-bus system with $\alpha = 0.01$, $\tau = 138.1$ with minimum placement and 462.2 with redundant placement. On an alarm, the control center computes the normalized residuals [31]

$$r_k^N = \frac{|r_k|}{\sqrt{\Omega_{kk}}}, \quad \Omega = R - HG^{-1}H^H, \quad (6)$$

with $R = W^{-1}$, removes all channels of the PMU that holds the largest one, estimates again and repeats the test, up to three PMUs. Removing every channel of that PMU matches the threat, since an attacker who controls a PMU controls all its channels.

Algorithm 2 Control center, one PDC set

Input: set S of received PMUs, H , σ , α , limits $[v_{\min}, v_{\max}]$, dead band δ , step Δv

- 1: $R \leftarrow \emptyset$ ▷ removed PMUs
- 2: estimate $\hat{V}$ from S by (4)
- 3: **if** S is not observable **then** add pseudo-measurements;
 $alarm \leftarrow false$
- 4: **else** compute J by (5) and τ ; $alarm \leftarrow J > \tau$
- 5: **while** $alarm$ and $|R| < 3$ and removal is enabled **do**
- 6: compute r^N by (6); $p \leftarrow$ PMU with the largest r_k^N
- 7: $R \leftarrow R \cup \{p\}$; drop all channels of p from S
- 8: re-estimate and re-test as in lines 2–4
- 9: **end while**
- 10: **if** $alarm$ **then**
- 11: **return** ▷ hold control while bad data remain
- 12: **end if**
- 13: **for** each bus i with $|\hat{V}_i| > v_{\max} + \delta$ or $|\hat{V}_i| < v_{\min} - \delta$ **do**
- 14: $g \leftarrow$ in-service generator nearest to i in branches
- 15: **if** no command was sent to g within the cooldown **then**
- 16: $v_g \leftarrow v_g \mp \Delta v$, clipped to the setpoint range
- 17: publish (g, v_g) on $cc/commands$
- 18: **end if**
- 19: **end for**

Voltage control. If the final estimate is free of alarms, the control center checks every bus against the limits $[v_{\min}, v_{\max}]$ with a dead band δ . For a bus outside the limits it picks the in-service generator with the fewest branches to that bus and moves its setpoint by one step in the direction that corrects the violation, within the setpoint range and at most once per cooldown period per generator. While an alarm remains after removal, no command is sent, because the estimate cannot be trusted. Algorithm 2 gives the procedure for one set.

F. Attack Models

The attacker targets a bus b and wants the control center to see $|V_b| = v_f$. In the linear model this is the state change $\Delta V = c e_b$ with $c = (v_f - |V_b|) e^{j\theta_b}$, which changes channel k by $h_{kb} c$. Four attacks act on the frames of the attacked PMUs between a start and an end time. The *simple FDI* changes one channel, the voltage channel of the PMU at b if there is one. The other channels still describe the true state, so the set becomes inconsistent. The *stealthy FDI* adds $a = H \Delta V$, that is $h_{kb} c$ to every channel with $h_{kb} \neq 0$. Then the residual is unchanged and so is J [6]. It needs every PMU that has such a channel. In both cases, an attacker who holds the PMU at b reads the true V_b from each frame and recomputes c , so the injection follows the grid. The *delay* attack holds the frames of every PMU with a channel that depends on V_b for an extra d_a , and the *drop* attack discards them. The attacker knows H and the operating point, which is the full-knowledge case of [6] and a bound on what the defenses can do.

G. Time Coupling and Closed-Loop Latency

All federates use the HELICS ZeroMQ core. The grid federate requests the times $j\Delta$. The network federate requests

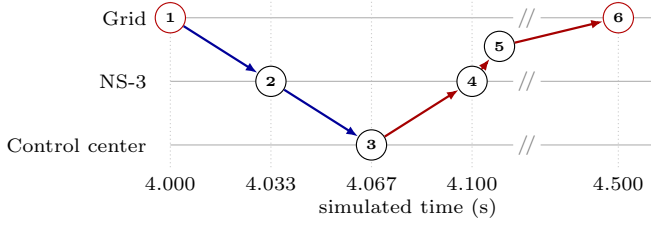


Fig. 4. Closed-loop timeline of the base experiment with the AVR fault at 4 s, as logged ($L = 10$ ms, $w = 20$ ms). (1) The grid applies the fault. (2) First PMU frame that carries it. (3) The control center receives the set and issues the command. (4) NS-3 reads the command. (5) The command reaches the generator node at 4.107 s. (6) The grid applies it at its next step.

the frame times kT one after another, runs the NS-3 event queue up to the next frame time, and at each granted time first publishes what the previous interval produced and then reads new phasors and commands. The control center does not step on its own. HELICS wakes it whenever a new PDC set is published. The grid and network federates are uninterruptible, so they are granted only the times they request.

This scheme fixes the latency of the loop. Suppose an event changes the grid at $t_0 = j\Delta$. Both the grid and the network federate are granted t_0 , and the network federate samples the phasors that were current before the grid published its new solution, so the event first appears in the frame stamped $t_0 + T$. If the PDC releases that set within one frame period, the network federate publishes it at $t_0 + 2T$, and the control center issues its command at that time. NS-3 reads the command at its next step and delivers it after the generator link delay d_g , and the grid applies it at its next step. The command therefore takes effect at

$$t_{\text{apply}} = \Delta \left\lceil \frac{t_0 + 3T + d_g + \epsilon}{\Delta} \right\rceil, \quad (7)$$

where ϵ is the processing and jitter delay. With $T = 33.3$ ms, $\Delta = 0.5$ s and d_g of a few tens of milliseconds, any violation the control center can see is corrected one grid step after it appears. Fig. 4 shows the timeline logged in the base experiment. The one-frame lag between grid and PMUs is a property of the time coupling. It could be removed by letting the network federate request a time just after the grid's, at the cost of a second synchronization per frame. We kept the simpler scheme and report the lag.

H. Operating Layer and Dashboard

An experiment is described by a scenario: grid, duration, MPI ranks, PMU placement and rate, network parameters, event, attack and control settings. Scenarios are entered in a form on the dashboard or uploaded as YAML files of up to 50 experiments that share common fields. The service checks each scenario against the chosen grid and rejects it with the name of the wrong field.

The service runs a set number of experiments at the same time, but starts an experiment only when its GridPACK ranks fit in the free MPI slots of the compute nodes. MPI ranks busy-wait for messages, so oversubscribed cores would slow every run that shares them. For each experiment the service

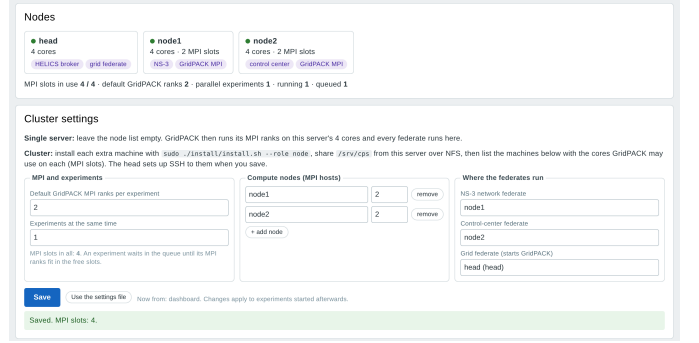


Fig. 5. Cluster page of the dashboard: nodes with their roles, MPI slots in use, the queue, and the settings for default ranks, experiments at the same time and the node of each federate.

creates a run directory on the shared file system, picks a free HELICS port, writes the configuration of each federate and starts the broker and the federates over the secure shell (SSH) on the nodes set on the Cluster page (Fig. 5). GridPACK's ranks are spread over the nodes through an MPI host file. If a federate fails, the others are stopped and the end of its log is shown in the queue.

The observer writes the state of the run about twice per second. The Live page shows the true and estimated voltages, the chi-square test, PDC completeness and each command. The Inside page shows what each engine computes, such as the partition of the MPI ranks, the Newton-Raphson mismatch, frames per PMU and the test before and after removal (Fig. 6). The Results page opens finished runs. Every run is a directory of plain files (frames, sets, estimates, commands, true voltages and summaries), so results can be analysed outside the dashboard. All figures and tables of Section VI were produced from these files by scripts that are published with the paper. The dashboard and the application programming interface (API) are protected by authentication, and remote users reach the head node through the institution's virtual private network (VPN) or an SSH tunnel.

V. IMPLEMENTATION AND EXPERIMENTAL SETUP

A. Test System and Preprocessing

All experiments use the IEEE 118-bus system, taken from the MATPOWER case file [58]. It has 118 buses, 186 branches and 54 generators. The testbed converts the case to GridPACK's input format and to a topology file used by the placement, the estimator and the network federate, and checks that both solvers agree on the base case before a grid is accepted. No other data set is involved. Every measurement in the study is generated by the co-simulation.

B. Hardware and Software

The testbed ran as a virtual cluster of three containers, a head node and two compute nodes with two MPI slots each, on one machine with four cores of an Intel Xeon processor at 2.1 GHz and 15 GB of memory. Fig. 7 shows the placement of each part, and Table IV lists the software. Two experiments ran at a time. The same build scripts install the testbed natively on Ubuntu or as a container image.

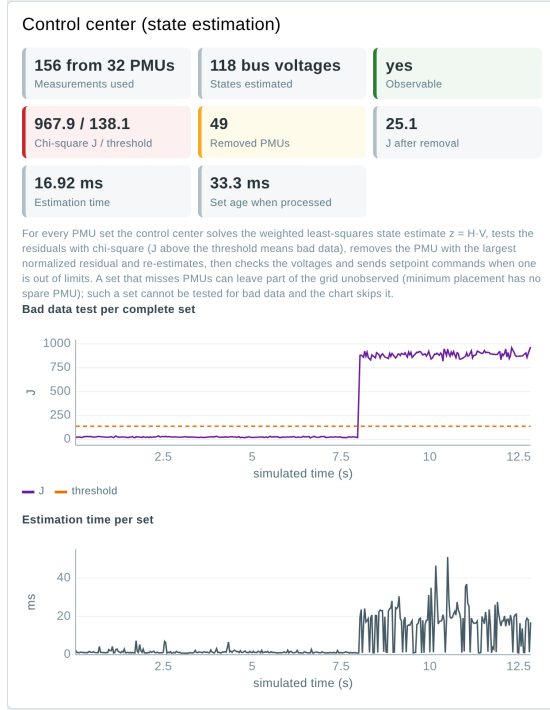


Fig. 6. Control-center panel of the Inside page during a simple FDI on bus 49 in a 30 s IEEE 118-bus run: measurements used, chi-square test before and after removing PMU 49, and estimation time per set.

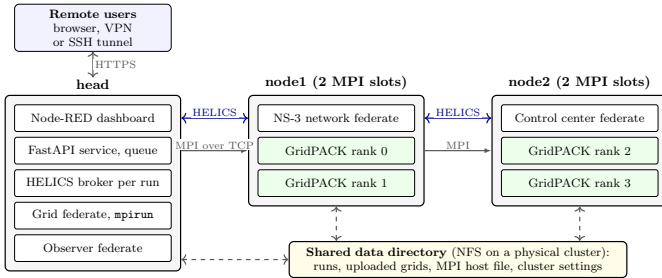


Fig. 7. Deployment on the three-node cluster used in this paper. GridPACK's ranks can span both compute nodes.

C. Scenarios and Parameters

The base scenario lasts 10 s with $\Delta = 0.5$ s, 30 frames per second, $L = 10$ ms, $w = 20$ ms, no random loss, and GridPACK on one MPI rank. At 4 s the AVR of generator 49 fails and moves its setpoint from 1.025 to 1.12 pu, which raises bus 49 above the upper limit of 1.08 pu. Every attack targets bus 49 from 4 to 9 s. Both injections try to make the control center see $|V_{49}| = 1.02$ pu, a normal value. Table V lists the parameters. From this base, 21 scenarios vary one factor at a time: the attack (none, simple FDI, stealthy FDI, delay of 100 ms, drop), the placement (32 or 68 PMUs), random frame loss (0 or 5%), the mean latency L (5, 10, 20, 40, 80 ms) and, at $L = 40$ ms, the wait window w (10, 20, 40, 60, 80 ms). Each scenario ran with seeds 1 to 10. The seed sets the link delays (1), the jitter, the loss, the measurement noise (2) and the random walk of the loads. Another 15 runs repeated the base scenario with 1, 2 and 4 MPI ranks and seeds 1 to 5. All 225 runs were defined in one scenario file and submitted

TABLE IV
SOFTWARE USED IN THE TESTBED

Component	Software
Grid simulation	GridPACK 3.5, PETSc 3.19, MUMPS, ParMETIS
Parallel runtime	Open MPI 4.1.6
Network simulation	NS-3.48
Co-simulation	HELICS 3.6.1, ZeroMQ core
Control center, API	Python 3.12, NumPy, SciPy, FastAPI
Dashboard	Node-RED 4.1, Dashboard 2.0
Operating system	Ubuntu 24.04

TABLE V
PARAMETERS OF THE BASE SCENARIO

Parameter	Value
Duration, grid step Δ	10 s, 0.5 s
PMU rate	30 frames/s
Mean link latency L , spread	10 ms, $\pm 50\%$
Processing delay, jitter	2 ms, $\mathcal{U}(0, 2)$ ms
PDC wait window w	20 ms
Noise	0.1% magnitude, 0.001 rad
Estimator σ , test level α	0.002, 0.01
Pseudo-measurement deviation	0.05 pu
Limits $[v_{\min}, v_{\max}]$, dead band δ	0.94–1.08 pu, 0.003 pu
Setpoint step, range, cooldown	0.01 pu, 0.95–1.10 pu, 1 s
Load variation	2% sinusoid, 20 s period, random walk
Event	AVR fault, generator 49, 1.12 pu at 4 s
Attack window, target, v_f	4–9 s, bus 49, 1.02 pu
Delay attack d_a	100 ms
GridPACK MPI ranks	1 (2 and 4 in Section VI-F)

through the API.

D. Baselines and Ablations

Two baselines bound the results. *No control* runs the same fault with the estimator and test but without voltage control, and gives the violation that the loop must remove. *Control without attack* gives the best the loop can do. The proposed configuration, with the chi-square test, removal of up to three PMUs and voltage control, is then compared with three ablations. *Detection without removal* keeps the test but holds control on every alarm, as a control center without identification would. *Minimum versus redundant placement* removes the extra PMUs. *PDC wait window* and *link latency* sweeps change the timing of the PDC and the network. For the computing layer, the MPI sweep compares one, two and four ranks.

E. Metrics and Statistical Procedure

For each run we record the *violation time*, the time during which the true voltage of some bus was outside the limits. It is measured at the grid steps and so has a resolution of 0.5 s. We also record the time the first command was issued, the share of PDC sets that were complete and the share that were observable, and the number of sets whose first test raised an alarm and the number still in alarm after removal. On the network side we record the share of frames that arrived in time, their latency, and the mean delay of the PDC release after

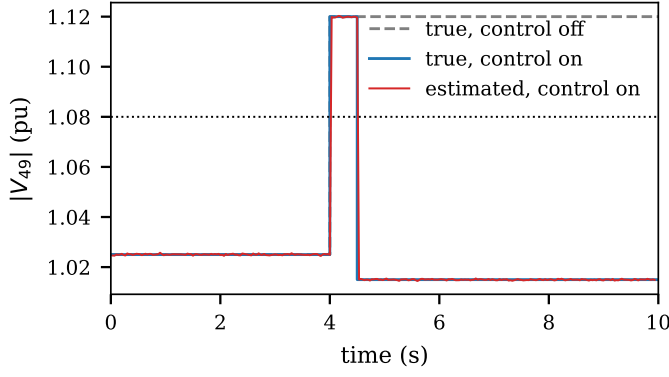


Fig. 8. Voltage of bus 49 without and with voltage control, and the control center's estimate (seed 1). The dotted line is the 1.08 pu limit.

the time stamp. Finally we record the mean absolute error of the estimate and the GridPACK solve time per step. For each scenario we report the mean and standard deviation over the ten seeds and a 95% confidence interval from Student's t distribution. Differences between a scenario and its reference are tested with a two-sided exact permutation test on the difference of means, which needs no assumption about the distribution and remains valid when many runs give the same value. We also give the difference of means with a 95% confidence interval. When all runs of both groups give the same value, the test cannot reject equality, and we say so.

VI. RESULTS

A. Validation of the Grid Solution

The base scenario was run once more with the testbed's built-in Newton–Raphson solver in place of GridPACK. At all 21 grid steps, the true voltages of the 118 buses were identical to the six decimals logged, both for GridPACK on one rank and for GridPACK on four ranks split over the two compute nodes. The partition, the MPI exchange and the restart-free server therefore do not change the solution.

B. Closed Loop Without Attack

Fig. 8 shows bus 49 in the two baselines. Without control the fault keeps $|V_{49}|$ near 1.12 pu to the end of the run, a violation of 6.5 s. With control, the command for generator 49 was issued at 4.067 s, delivered at 4.107 s and applied at 4.5 s, as predicted by (7) and shown in Fig. 4. It lowered the setpoint to 1.015 pu, and the violation lasted one grid step, 0.5 s. The same times were logged in all ten seeds. All 9568 frames of each run arrived in time, with a median latency of 12.8 ms and a 95th percentile of 17.3 ms (means over seeds), and the PDC released each set 17.9 ms after its time stamp on average. The mean absolute error of the estimate was 2.8×10^{-4} pu, and no set raised an alarm.

C. Attacks and PMU Placement

Table VI lists the 13 scenarios on attacks and placement, and Fig. 9 shows the violation time with its 95% confidence interval. In these scenarios the random factors changed the

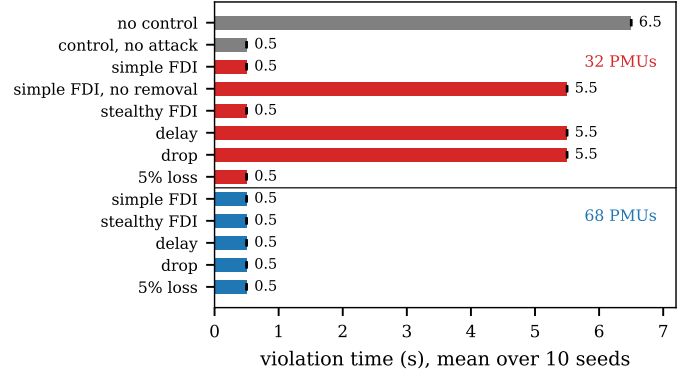


Fig. 9. Violation time per scenario, mean over ten seeds with 95% confidence interval. Red: 32 PMUs; blue: 68 PMUs; grey: baselines.

frames, delays and noise but never the outcome, so every seed gave the same violation time and the same command time.

Simple FDI with 32 PMUs. The attacker holds PMU 49 and replaces its voltage channel. In the set stamped 4.000 s the fault is not yet visible and the falsified 1.02 pu is close to the true 1.025 pu, so $J = 36$ (seed 1). In the next set the true voltage is 1.12 pu, the channel still reads 1.02 pu, and the current channels disagree with it. J rose to 2414 against a threshold of 138.1 (Fig. 11). The largest normalized residual pointed to PMU 49, which was removed, and the new estimate put bus 49 at 1.119 pu (Fig. 12). The command was issued at 4.067 s, as without attack. After the command took effect, the true voltage was 1.015 pu, close to the value the attacker reported, so J fell below the threshold for the rest of the attack. This is why exactly 15 sets, $0.5 \text{ s} \times 30 \text{ sets/s}$, raised an alarm in every seed. Removing PMU 49 left some buses without an observing PMU in those 15 sets, so the final estimate could not be tested again. The control center acted on an estimate completed with pseudo-measurements, which explains the 95.0% of observable sets. The command was correct, but the test that would have confirmed it was not run.

Detection without removal. With the same attack and the test in place but no removal, every set from 4.033 to 9.0 s raised an alarm. The estimate of bus 49 stayed near the truth, at 1.116 pu on average in seed 1, because the current channels outweighed the single false voltage channel. The control center nevertheless held its commands while the alarm stood. The command came only after the attack ended, at 9.033 s, and the violation lasted 5.5 s in every seed. Detection without identification protected the estimate and blocked the control.

Stealthy FDI. Bus 49 is observed by PMU 49 and by the current that PMU 45 measures on branch 45–49, so the stealthy attack changed both. As expected, J did not move and stayed below 36 during the attack (Fig. 11), and the estimate of bus 49 stayed at 1.02 pu (Fig. 10). The attack still failed to hide the fault's effect on the neighbouring buses, whose measurements are untouched. The control center estimated bus 48 at 1.096 pu, issued the command because of that bus at 4.067 s, and the command went to generator 49, the generator nearest to bus 48.

TABLE VI
ATTACKS AND PMU PLACEMENT, IEEE 118-BUS SYSTEM, AVR FAULT AT GENERATOR 49 AT 4 s, ATTACKS ON BUS 49 FROM 4 TO 9 s. MEAN $\pm$ STANDARD DEVIATION OVER TEN SEEDS; A SINGLE VALUE MEANS ALL SEEDS GAVE THE SAME RESULT

Case	Attacked PMUs (bus)	Complete sets (%)	Observable sets (%)	Alarm sets (first test)	Alarm sets (after removal)	First command issued (s)	Violation time (s)
<i>Minimum placement, 32 PMUs</i>							
No control (baseline)	–	100.0	100.0	0	0	–	6.5
Control, no attack (baseline)	–	100.0	100.0	0	0	4.067	0.5
Simple FDI	49	100.0	95.0	15	0	4.067	0.5
Simple FDI, no removal	49	100.0	100.0	149	149	9.033	5.5
Stealthy FDI	45, 49	100.0	100.0	0	0	4.067	0.5
Delay 100 ms	45, 49	49.7	49.7	0	0	9.033	5.5
Drop	45, 49	49.7	49.7	0	0	9.033	5.5
5% frame loss	–	20.5 $\pm$ 1.5	20.5 $\pm$ 1.5	0	0	4.067	0.5
<i>Redundant placement, 68 PMUs</i>							
Simple FDI	49	100.0	100.0	15	0	4.067	0.5
Stealthy FDI	42, 45, 49, 51, 66	100.0	100.0	0	0	4.067	0.5
Delay 100 ms	42, 45, 49, 51, 66	49.7	49.7	0	0	4.067	0.5
Drop	42, 45, 49, 51, 66	49.7	49.7	0	0	4.067	0.5
5% frame loss	–	3.0 $\pm$ 0.8	85.3 $\pm$ 2.0	0	0	4.067	0.5

Delay and drop. Delaying the frames of PMUs 45 and 49 by 100 ms, five times the wait window, made them late for every set. Their sets were released without them, so 50.3% of the sets were incomplete and unobservable. Bus 49 was then estimated from pseudo-measurements of the last value seen before the attack, 1.025 pu (Fig. 10), and the violation went unseen until the frames returned at 9 s, giving 5.5 s of violation in every seed. Dropping the frames had the same effect. For the control center, a frame that arrives after the window is a lost frame.

Redundant placement. With 68 PMUs the simple FDI was detected in the same set and PMU 49 was removed. This time the grid stayed observable after removal and the second test passed ($J = 77$ against 434 in seed 1). The stealthy FDI now needed five PMUs and was again not detected, but the neighbours exposed the fault. Under delay and drop of all five PMUs that observe bus 49, bus 48 remained observed through other PMUs and was estimated at 1.096 pu, and the command was issued at 4.067 s. Every attack in the set left the violation at 0.5 s.

Random loss. With 5% of frames lost on every link, only 20.5% of the 32-PMU sets were complete. Because the loss was spread over all PMUs, the frames of bus 49's PMUs arrived in most sets, and the violation was corrected as without loss. With 68 PMUs only 3.0% of the sets were complete, since a set needs all 68 frames, but 85.3% were still observable, against 20.5% with 32 PMUs.

D. Network Latency and the PDC Wait Window

Table VII and Fig. 13 repeat the controlled run without attack at other mean latencies. Because each link delay is drawn between $0.5L$ and $1.5L$ by (1), the spread of delays grows with L , while the wait window is measured from the first frame of a time stamp. Once the spread exceeds w , the slow links miss their sets. At $L = 20$ ms the share of complete sets fell to $67.8 \pm 26.9\%$ and depended strongly on the seed, between 28% and 100%, while 98.6% of the frames still arrived in time. At $L = 40$ ms no set was complete

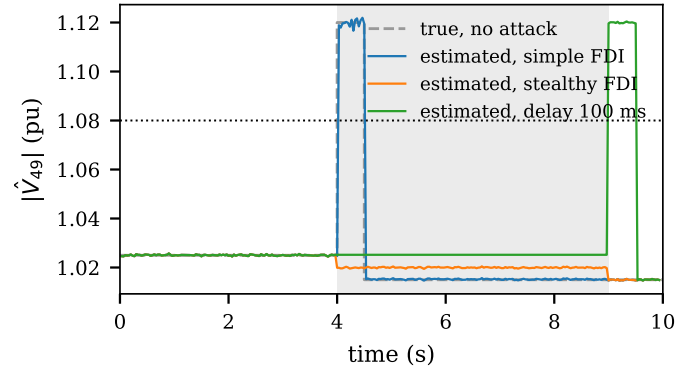


Fig. 10. Estimate of bus 49 under three attacks with 32 PMUs (seed 1). Under the simple FDI the estimate follows the true voltage. Shaded: attack window.

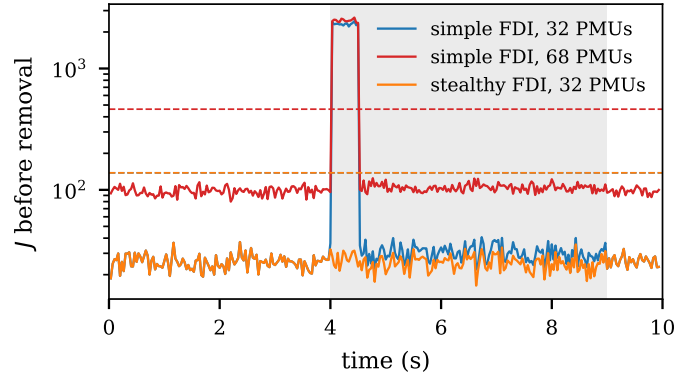


Fig. 11. Chi-square statistic before removal (solid) and threshold (dashed) for each case, seed 1. The simple FDI is detected from 4.033 to 4.5 s with both placements. The stealthy FDI leaves J unchanged. Shaded: attack window.

or observable in any seed, and whether the fault was seen depended on the delays drawn for PMUs 45 and 49. In seed 1, PMU 49's link had 32 ms against a fastest link of 21 ms, its frames fit in the window, and the command was issued at 4.100 s. In seed 3 its link had 49.5 ms against 20.1 ms, its frames were always late, and the violation lasted to the end of

TABLE VII

EFFECT OF MEAN LINK LATENCY L AND PDC WAIT WINDOW w , IEEE 118-BUS SYSTEM, MINIMUM PLACEMENT, AVR FAULT, NO ATTACK. MEAN $\pm$ STANDARD DEVIATION OVER TEN SEEDS

L (ms)	w (ms)	Frames in time (%)	Complete sets (%)	Observable sets (%)	Latency p50 (ms)	Latency p95 (ms)	PDC release delay (ms)	Estimation error (10^{-3} pu)	Violation time (s)
5	20	100.0	100.0	100.0	7.8 ± 0.4	10.2 ± 0.3	10.5 ± 0.1	0.28 ± 0.00	0.5
10	20	100.0	100.0	100.0	12.8 ± 1.0	17.3 ± 0.3	17.9 ± 0.2	0.28 ± 0.00	0.5
20	20	98.6 ± 1.3	67.8 ± 26.9	67.8 ± 26.9	22.8 ± 1.8	31.5 ± 0.7	32.5 ± 0.2	0.29 ± 0.02	0.5
40	20	53.1 ± 9.1	0.0	0.0	34.2 ± 2.2	42.7 ± 1.7	44.0 ± 1.8	9.07 ± 2.67	4.1 ± 3.1
80	20	25.7 ± 8.2	0.0	0.0	53.9 ± 4.6	62.1 ± 5.0	65.4 ± 3.5	16.33 ± 2.00	5.9 ± 1.9
40	10	25.8 ± 8.2	0.0	0.0	28.3 ± 2.3	32.5 ± 2.5	34.0 ± 1.8	15.99 ± 2.09	5.9 ± 1.9
40	40	99.1 ± 0.9	85.5 ± 19.3	85.5 ± 19.3	43.2 ± 3.7	60.7 ± 1.1	62.2 ± 0.4	0.29 ± 0.01	0.5
40	60	99.7	100.0	100.0	43.3 ± 3.7	60.9 ± 1.1	62.2 ± 0.4	0.28 ± 0.00	0.5
40	80	99.7	100.0	100.0	43.3 ± 3.7	60.9 ± 1.1	62.2 ± 0.4	0.28 ± 0.00	0.5

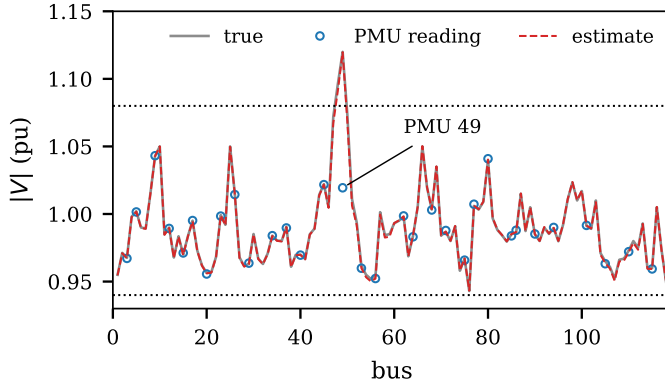


Fig. 12. Voltage of every bus in the set stamped 4.1 s, simple FDI on bus 49, 32 PMUs, seed 1: true voltage of the state the PMUs sampled, PMU readings, and the estimate after PMU 49 was removed. PMU 49 reported 1.019 pu for a true 1.120 pu.

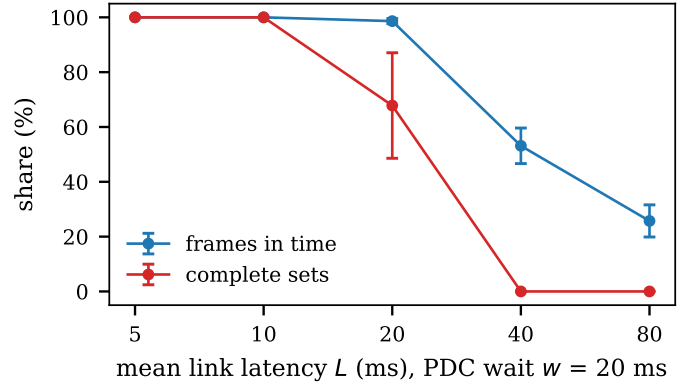


Fig. 13. Share of frames that arrived in time and of complete PDC sets against the mean link latency, $w = 20$ ms, mean over ten seeds with 95% confidence interval.

the run. Over ten seeds the fault went unseen in six, and the violation averaged 4.1 s with no attack at all. At $L = 80$ ms the fault went unseen in nine seeds out of ten. The mean absolute error of the estimate rose from 2.8×10^{-4} pu to 9.1×10^{-3} pu at 40 ms and 1.6×10^{-2} pu at 80 ms, because the unobserved buses were held near earlier values.

Fig. 14 shows the remedy at $L = 40$ ms. A wait window of 10 ms made matters worse, with the fault unseen in nine seeds. A window of 40 ms, equal to the expected spread L , restored $85.5 \pm 19.3\%$ complete sets and corrected the fault in every seed, and 60 ms restored all sets. The price is latency. The PDC released sets 62 ms after their time stamp with $w \geq 40$ ms, against 18 ms at $L = 10$ ms. Beyond 60 ms, a longer window brought nothing, because every frame had already arrived. The wait window therefore has to follow the spread of the link delays, and their mean is a poor guide.

E. Ablation and Statistical Significance

Table VIII tests the main comparisons. Removing the bad PMU, adding redundant PMUs against delay and drop, and closing the voltage loop each changed the violation time by 5.0 to 6.0 s with $p = 1.1 \times 10^{-5}$, the smallest value an exact two-sided test can give with ten runs per group, since all twenty runs fell on the expected side. The simple FDI with removal and the stealthy FDI did not change the violation

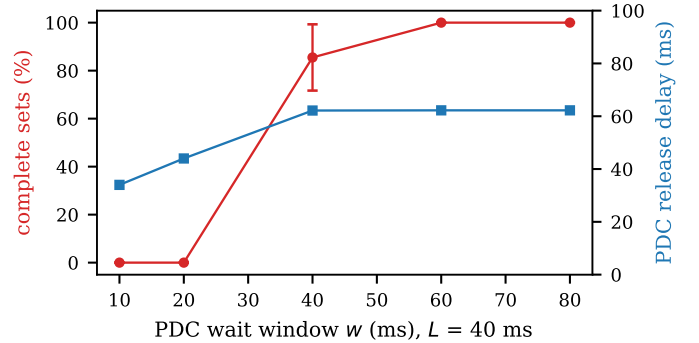


Fig. 14. Complete sets (left axis) and mean PDC release delay after the time stamp (right axis) against the PDC wait window at $L = 40$ ms, mean over ten seeds with 95% confidence interval.

time compared with the attack-free run in any seed, so the test cannot reject equality ($p = 1$). For the network, the drop in complete sets from 10 to 20 ms ($p = 0.003$), the gain from a 40 ms instead of a 20 ms window at $L = 40$ ms, and the longer violations at 80 ms latency or with a 10 ms window ($p = 1.2 \times 10^{-4}$) were all significant at the 1% level. Redundant placement under 5% loss raised the observable sets by 64.8 percentage points (95% confidence interval 63.1 to 66.4).

TABLE VIII
STATISTICAL TESTS BETWEEN CONFIGURATIONS, TEN SEEDS EACH.
DIFFERENCE OF MEANS (B – A) WITH 95% CONFIDENCE INTERVAL
AND TWO-SIDED EXACT PERMUTATION TEST

Comparison (A vs. B)	Mean A	Mean B – A	p
Voltage control (no control vs. control)	0.5 s	+6.0 s	1.1×10^{-5}
Removal (simple FDI, 32 PMUs)	0.5 s	+5.0 s	1.1×10^{-5}
Simple FDI with removal vs. no attack	0.5 s	+0.0 s	1
Stealthy FDI vs. no attack (32 PMUs)	0.5 s	+0.0 s	1
Placement under delay	0.5 s	+5.0 s	1.1×10^{-5}
Placement under drop	0.5 s	+5.0 s	1.1×10^{-5}
Placement under 5% loss (observable sets)	85.3 %	-64.8 [-66.4, -63.1] %	1.1×10^{-5}
Latency 10 vs. 20 ms (complete sets)	100.0 %	-32.2 [-50.0, -14.3] %	0.003
Wait 40 vs. 20 ms at $L = 40$ ms (complete sets)	85.5 %	-85.5 [-98.3, -72.7] %	1.1×10^{-5}
Wait 40 vs. 10 ms at $L = 40$ ms (violation)	0.5 s	+5.4 [+4.1, +6.7] s	1.2×10^{-4}
Latency 10 vs. 80 ms (violation)	0.5 s	+5.4 [+4.1, +6.7] s	1.2×10^{-4}

TABLE IX
COMPUTING TIME PER EXPERIMENT WITH 1, 2 AND 4 GRIDPACK MPI RANKS, BASE SCENARIO, FIVE SEEDS EACH (MEAN $\pm$ STANDARD DEVIATION)

Ranks	Solve time (ms)	Grid step (ms)	Wall time (s)
1	9.6 ± 1.6	17.5 ± 4.2	4.8 ± 0.4
2	32.6 ± 10.5	52.1 ± 18.7	6.1 ± 0.6
4	90.5 ± 19.5	136.0 ± 15.1	7.3 ± 0.3

F. Computing Time

Table IX gives the GridPACK time per grid step and the wall-clock time of a 10 s experiment with one, two and four ranks, while a second experiment ran at the same time. A 118-bus power flow takes about 10 ms on one rank, so every additional rank adds more communication than it removes work, and four ranks split over two containers that exchange MPI messages over the transmission control protocol (TCP) needed 90 ms per solve. For this grid one rank is the right choice, and the cluster pays off by running many experiments at once. The runs of the study took between 4.0 and 9.0 s each, except the first two after the cluster started, which took 20 s.

Table X gives the control center's time per set, measured by a benchmark of the estimator on the IEEE 118-bus system with 200 noisy sets per case. A clean set took a median of 0.9 ms with either placement. When an alarm triggered the normalized-residual test, which needs one solve per channel, and a second estimate, the median rose to 19 ms with 32 PMUs and 38 ms with 68 PMUs, with a maximum of 57 ms. These times do not affect the simulated time of a run, but they matter for a real-time version, because a frame period is 33.3 ms.

G. Comparison With Prior Work

Direct numerical comparison with the platforms of Table I is limited, because they study other systems, events and metrics.

TABLE X
CONTROL-CENTER COMPUTING TIME PER PDC SET, IEEE 118-BUS SYSTEM

PMUs	Set	Median (ms)	p95 (ms)	Max (ms)
32	clean	0.91	1.50	3.90
32	FDI, PMU 49 removed	19.39	29.58	37.32
68	clean	0.85	1.53	2.30
68	FDI, PMU 49 removed	37.60	51.46	56.84

Where the quantities match, the results agree. The median PMU-to-PDC latencies of 12.8 ms at $L = 10$ ms and 22.8 ms at $L = 20$ ms bracket the mean of about 20 ms that Morato *et al.* found for PMU traffic over 5G [30]. Our earlier IEEE 14-bus prototype reported latencies rising from 10 to about 100 ms under denial of service but could not show the effect on the grid [24]. The present study shows that a delay of that order hides an over-voltage for 5.5 s under minimum placement. Zheng *et al.* found that GridPACK under HELICS runs faster than real time for a 3000-bus system on four processes [23]. Our single-rank runs simulated 10 s of a smaller system in about 5 s of wall-clock time on shared cores, with the full network and control center in the loop. The value of the chi-square test against stealthy attacks matches [6], [34]: it did not react at all. The benefit of placement that keeps the grid observable without the attacked PMUs matches the analysis of [37], here shown in a closed loop with a simulated network.

Calibration of the test. The chi-square statistic in normal operation averaged 25.5 with minimum placement, between 24.7 and 26.3 over the seeds, well below its 102 degrees of freedom. The estimator assumes $\sigma = 0.002$ while the simulated noise is 0.001 in magnitude and 0.001 rad in angle, so each real component of a residual has about a quarter of the variance that the test assumes, and $(2m - 2n)/4 \approx 26$. The test raised no false alarm in any of the 225 runs.

VII. DISCUSSION

A. Interpretation

The experiments separate the defenses by what they protect. The chi-square test with normalized-residual removal protects the estimate, and with it the control action, against a falsified channel, provided enough redundancy remains after removal to test the result again. Detection alone protects the estimate but blocks the control, which in this case cost five seconds of over-voltage in every seed. The stealthy injection did not raise the test at all, and it failed here only because it shifted one bus while the fault also raised the neighbouring buses. An attacker who falsifies the neighbourhood too would need more PMUs, and the learned detectors of [9], [10] are one way to catch such attacks. Under minimum placement, delay and drop were the most effective attacks. Neither changes a value, and both turn a measurement into a missing one, which is consistent with the role of data dropout in time-synchronization attacks [11], [12]. Redundant placement neutralized them because it kept the neighbourhood of bus 49 observed, which is the condition Khalafi *et al.* derived for locating attacks [37]. The PDC results agree with the observation that wait times must be set from

the latency of the slowest stream [5]. A frame that arrives after the window is, for the control center, a lost frame.

B. Limitations

The cluster was virtual, three containers on four cores, so the MPI times include contention and TCP between containers and do not show what separate machines would give. The grid runs in quasi-steady state with a 0.5 s step, which sets the resolution of the violation time and the delay before a command takes effect. The testbed also has a dynamic mode, which this paper does not use. The frames are JSON and do not follow IEEE C37.118.2, so real PMUs cannot yet join an experiment. The co-simulation runs as fast as it can and is not paced to wall-clock time. The PMU frames lag the grid by one frame, as explained in Section IV-G. The control center acts on an estimate once its alarm has cleared, even when the cleared estimate could not be tested because removal made the grid unobservable.

C. Threats to Validity

Internal validity. The outcome depends on the time coupling. We derived the loop latency in (7) and checked it against the logged times of every command. The grid solution was checked against an independent Newton–Raphson solver. Each scenario changed one factor from the base, and all runs used the same software build.

Construct validity. The violation time is measured at grid steps, so differences smaller than 0.5 s are not visible. The chi-square test assumes $\sigma = 0.002$ while the simulated noise is smaller (Section VI), which makes the test conservative. A smaller injection would be harder to detect than with matched weights.

Statistical validity. Ten seeds per scenario vary noise, delays, loss and loads but not the fault or the attack, which are fixed by design. Many outcomes were therefore identical in all seeds. The tests show that the differences between these groups cannot be explained by the random factors, but they do not cover other faults, targets or attack sizes.

External validity. All results come from one system, one fault and one target bus. Bus 49 is a well-connected generator bus. A bus with fewer neighbours, a different placement or a dynamic event could change the outcome, and the testbed can run those cases from the same scenario file.

VIII. CONCLUSION AND FUTURE WORK

This paper presented an open-source testbed in which Grid-PACK on MPI, NS-3 with a PMU network and a PDC, and a control center with state estimation, bad data detection and voltage control run as HELICS federates, and in which remote users start, follow and compare experiments from a browser on a shared cluster. On the IEEE 118-bus system, with one AVR fault and four attacks on the PMUs around the faulted bus, the closed loop corrected the over-voltage within one grid step whenever the control center could see it. Ten seeds per scenario showed the same pattern each time. Removing the suspect PMU was as important as detecting the attack. Delay

and drop hid the fault under minimum placement and failed under redundant placement. The PDC wait window had to cover the spread of link delays, or the sets lost completeness and observability even when most frames arrived.

Future work will add an IEEE C37.118.2 interface so that the project's PMUs can join experiments, run the testbed on the laboratory's physical cluster, use the dynamic mode to study fast events and wide-area control, add authenticated frames and learned detectors to the control center, and pace the co-simulation to wall-clock time. The source code, installer, scenario files and the scripts that produced every figure and table are available at <https://github.com/mazharulmd/cps-testbed-cluster>.

ACKNOWLEDGMENT

The authors thank the Bangladesh Energy and Power Research Council for funding this work under grant EPRC/58-2018-007-01.

REFERENCES

- [1] A. Abur and A. Gómez Expósito, *Power System State Estimation: Theory and Implementation*. New York, NY, USA: Marcel Dekker, 2004.
- [2] A. G. Phadke and J. S. Thorp, *Synchronized Phasor Measurements and Their Applications*. New York, NY, USA: Springer, 2008.
- [3] "IEEE standard for synchrophasor measurements for power systems," IEEE Std C37.118.1-2011, 2011.
- [4] "IEEE guide for phasor data concentrator requirements for power system protection, control, and monitoring," IEEE Std C37.244-2013, 2013.
- [5] R. Pourramezan, H. Karimi, J. Mahseredjian, and M. Paolone, "Optimal coordination of phasor data concentrators in hierarchical synchrophasor networks," *IEEE Trans. Smart Grid*, vol. 12, no. 3, pp. 2402–2412, 2021.
- [6] Y. Liu, P. Ning, and M. K. Reiter, "False data injection attacks against state estimation in electric power grids," in *Proc. 16th ACM Conf. Comput. Commun. Secur. (CCS)*, 2009, pp. 21–32.
- [7] O. Kosut, L. Jia, R. J. Thomas, and L. Tong, "Malicious data attacks on the smart grid," *IEEE Trans. Smart Grid*, vol. 2, no. 4, pp. 645–658, 2011.
- [8] R. B. Bobba, K. M. Rogers, Q. Wang, H. Khurana, K. Nahrstedt, and T. J. Overbye, "Detecting false data injection attacks on DC state estimation," in *Proc. 1st Workshop Secure Control Syst. (SCS)*, Stockholm, Sweden, 2010.
- [9] C. Chen, Y. Wang, M. Cui, J. Zhao, W. Bi, Y. Chen, and X. Zhang, "Data-driven detection of stealthy false data injection attack against power system state estimation," *IEEE Trans. Ind. Informat.*, vol. 18, no. 12, pp. 8467–8476, 2022.
- [10] A. Bhattacharjee, A. K. Mondal, A. Verma, S. Mishra, and T. K. Saha, "Deep latent space clustering for detection of stealthy false data injection attacks against AC state estimation in power systems," *IEEE Trans. Smart Grid*, vol. 14, no. 3, pp. 2338–2351, 2023.
- [11] M. Zadsar *et al.*, "Preventing time-synchronization attacks on synchrophasor measurements of wide-area damping controllers," *IEEE Trans. Instrum. Meas.*, 2023.
- [12] E. Shereen and G. Dán, "Network topology-aware mitigation of undetectable PMU time synchronization attacks," *IEEE Trans. Control Netw. Syst.*, 2024.
- [13] K. Kumar, P. Saini, A. Prakash, S. K. Parida, K. A. Al Jaafari, H. H. Zeineldin, and E. F. El-Saadany, "Real-time cyber-physical co-simulation for resilient wide-area damping control against FDIAs and communication disruptions," *IEEE Trans. Ind. Appl.*, vol. 61, no. 6, 2025.
- [14] T.-T. Nguyen, R. Kadavil, and H. Hooshyar, "A real-time cyber-physical simulation testbed for cybersecurity assessment of large-scale power systems," *IEEE Trans. Ind. Appl.*, vol. 60, no. 6, pp. 8329–8340, 2024.
- [15] S. C. Das, T. Vu, and H. Ginn, "Scalable cyber-physical testbed for cybersecurity evaluation of synchrophasors in power systems," *IET Cyber-Phys. Syst.: Theory Appl.*, 2025.
- [16] D. Mishchenko, I. Oleinikova, L. Erdődi, and B. R. Pokhrel, "Multidomain cyber-physical testbed for power system vulnerability assessment," *IEEE Access*, vol. 12, pp. 38 135–38 149, 2024.

- [17] B. Palmintier, D. Krishnamurthy, P. Top, S. Smith, J. Daily, and J. Fuller, "Design of the HELICS high-performance transmission-distribution-communication-market co-simulation framework," in *Proc. Workshop Model. Simul. Cyber-Phys. Energy Syst. (MSCPES)*, Pittsburgh, PA, USA, 2017, pp. 1–6.
- [18] T. D. Hardy, B. Palmintier, P. L. Top, D. Krishnamurthy, and J. C. Fuller, "HELICS: A co-simulation framework for scalable multi-domain modeling and analysis," *IEEE Access*, vol. 12, pp. 24 325–24 347, 2024.
- [19] O. Bel, J. Kim, W. J. Hofer, M. Maharjan, B. Hyder, S. Purohit, and S. Niddodi, "Co-simulation framework for network attack generation and monitoring," *IEEE Access*, vol. 12, 2024.
- [20] N. Thioub and M. Adda, "GridSecSim: A hybrid cyber-physical co-simulation platform integrating GridLAB-D and NS-3 for cyberattack simulation and real-time intrusion detection in smart grids," *IEEE Access*, vol. 14, 2026.
- [21] S. C. Vallam Kondu, V. R. Motakatla, F. Ding, and J. Zhang, "A high-fidelity cyber-physical DER system testbed for analyzing DER monitoring and control under cyberattacks," in *Proc. IEEE Conf.*, 2025.
- [22] M. Y. Masood, H. Rifā-Pous, M. Awais, A. Khawar, and M. Muzaffar, "Co-simulation of FDI and DoS attacks in IoT-enabled next-generation smart grids: A framework for resilience," in *Proc. 7th Int. Conf. Adv. Comput. Sci. (ICACS)*, 2026.
- [23] L. Zheng, Y. Cui, S. Jin, and Y. Chen, "High-performance computing-based open-source power transmission and distribution grid co-simulation," *IEEE Trans. Power Syst.*, vol. 39, no. 5, pp. 6144–6153, 2024.
- [24] M. Islam, T. A. Khan, S. Zulfiker, A. S. M. J. Hasan, and H. A. Rahman, "Remotely accessible cyber-physical system testbed for power grid's security and reliability," in *Proc. 8th Int. Conf. Smart Grid Smart Cities (ICSGSC)*, 2024, pp. 436–441.
- [25] B. Palmer, W. Perkins, Y. Chen, S. Jin, D. Callahan, K. Glass, R. Diao, M. Rice, S. Elbert, M. Vallem, and Z. Huang, "GridPACK: A framework for developing power grid simulations on high-performance computing platforms," *Int. J. High Perform. Comput. Appl.*, vol. 30, no. 2, pp. 223–240, 2016.
- [26] G. F. Riley and T. R. Henderson, "The ns-3 network simulator," in *Modeling and Tools for Network Simulation*, K. Wehrle, M. Güneş, and J. Gross, Eds. Berlin, Germany: Springer, 2010, pp. 15–34.
- [27] OpenJS Foundation, "Node-RED," [Online]. Available: <https://nodered.org>, 2024.
- [28] "IEEE standard for synchrophasor data transfer for power systems," IEEE Std C37.118.2-2011, 2011.
- [29] P. Castello, G. Gallus, P. A. Pegoraro, and S. Sulis, "Measurement platform for latency characterization of wide area monitoring, protection and control systems," *IEEE Trans. Instrum. Meas.*, vol. 73, p. 5500812, 2024.
- [30] A. Morato, G. Frigo, and F. Tramarin, "5G-enabled PMU-based distributed measurement systems: Network infrastructure optimization and scalability analysis," *IEEE Trans. Instrum. Meas.*, vol. 73, 2024.
- [31] E. Handschin, F. C. Schweppe, J. Kohlas, and A. Fiechter, "Bad data analysis for power system state estimation," *IEEE Trans. Power App. Syst.*, vol. PAS-94, no. 2, pp. 329–337, 1975.
- [32] T. L. Baldwin, L. Mili, M. B. Boisen, and R. Adapa, "Power system observability with minimal phasor measurement placement," *IEEE Trans. Power Syst.*, vol. 8, no. 2, pp. 707–715, 1993.
- [33] B. Gou, "Generalized integer linear programming formulation for optimal PMU placement," *IEEE Trans. Power Syst.*, vol. 23, no. 3, pp. 1099–1104, 2008.
- [34] H. Yang, W. Zhang, C. Y. Chung, Z. Wang, W. Qiu, and Z. Liang, "AC false data injection attack based on robust tensor principle component analysis," *IEEE Trans. Ind. Informat.*, 2024.
- [35] M. Zadsar, M. Ghafouri, A. Ameli, and B. Moussa, "Integrated deep learning-based detection and robust state observer mitigation of time-synchronization attacks: A WADC case study," *IEEE Trans. Smart Grid*, vol. 15, no. 5, pp. 5055–5071, 2024.
- [36] K. P. Swain, A. Sharma, A. Karkare, S. Chakrabarti, E. Gryazina, and V. Terzija, "Network-level vulnerability assessment of synchrophasor measurement devices," *IEEE Access*, vol. 12, pp. 72 491–72 503, 2024.
- [37] Z. S. Khalafi, M. Dehghani, A. Khalili, A. Sami, N. Vafamand, and T. Dragičević, "Intrusion detection, measurement correction, and attack localization of PMU networks," *IEEE Trans. Ind. Electron.*, vol. 69, no. 5, pp. 4697–4706, 2022.
- [38] Z. Pourahmad *et al.*, "Optimal placement of PMU and PDC in power systems by considering the vulnerabilities against cyber-attacks," *Elect. Eng.*, 2023.
- [39] K. Hopkinson, X. Wang, R. Giovanini, J. Thorp, K. Birman, and D. Coury, "EPOCHS: A platform for agent-based electric power and communication simulation built from commercial off-the-shelf components," *IEEE Trans. Power Syst.*, vol. 21, no. 2, pp. 548–558, 2006.
- [40] H. Lin, S. S. Veda, S. S. Shukla, L. Mili, and J. Thorp, "GECO: Global event-driven co-simulation framework for interconnected power system and communication network," *IEEE Trans. Smart Grid*, vol. 3, no. 3, pp. 1444–1456, 2012.
- [41] K. Mets, J. A. Ojea, and C. Develder, "Combining power and communication network simulation for cost-effective smart grid analysis," *IEEE Commun. Surveys Tuts.*, vol. 16, no. 3, pp. 1771–1796, 2014.
- [42] S. Schütte, S. Scherfke, and M. Tröschel, "Mosaik: A framework for modular simulation of active components in smart grids," in *Proc. IEEE 1st Int. Workshop Smart Grid Model. Simul. (SGMS)*, 2011, pp. 55–60.
- [43] D. Qiu, M. Liu, R. Zhang, T. Luo, A. Griffo, and X. Zhang, "Cyber-physical real-time digital simulation for cybersecurity analysis in micro-grids," *IEEE Trans. Ind. Cyber-Phys. Syst.*, vol. 3, pp. 429–441, 2025.
- [44] A. H. Dabashi, D. Qiu, X. Zhang, and G. Taylor, "Innovative testbed configurations and interfacing techniques for real-time digital simulation of cyber-physical energy and power systems," *Electronics*, vol. 15, p. 3705, 2026.
- [45] A. Hahn, A. Ashok, S. Sridhar, and M. Govindarasu, "Cyber-physical security testbeds: Architecture, application, and evaluation for smart grid," *IEEE Trans. Smart Grid*, vol. 4, no. 2, pp. 847–855, 2013.
- [46] A. Ashok, S. Krishnaswamy, and M. Govindarasu, "PowerCyber: A remotely accessible testbed for cyber physical security of the smart grid," in *Proc. IEEE PES Innov. Smart Grid Technol. Conf. (ISGT)*, Minneapolis, MN, USA, 2016, pp. 1–5.
- [47] V. K. Singh, M. Govindarasu, D. Porsch, E. Shaffer, and M. Berman, "NEFTSec: Networked federation testbed for cyber-physical security of smart grid: Architecture, applications, and evaluation," *IET Cyber-Phys. Syst.: Theory Appl.*, vol. 7, no. 4, pp. 197–211, 2022.
- [48] M. S. Abdelrahman, I. Kharchouf, T. L. Nguyen, and O. A. Mohammed, "A hybrid physical co-simulation smart grid testbed for testing and impact analysis of cyber-attacks on power systems: Framework and attack scenarios," *Energies*, vol. 16, no. 23, p. 7771, 2023.
- [49] M. Chamana, R. Bhatta, K. Schmitt, R. Shrestha, and S. Bayne, "An integrated testbed for power system cyber-physical operations training," *Appl. Sci.*, vol. 13, no. 16, p. 9451, 2023.
- [50] X. Wang *et al.*, "Improving cyber-physical-power system stability through hardware-in-loop co-simulation platform for real-time cyber attack analysis," *Front. Energy Res.*, vol. 12, 2024.
- [51] B. Chng, B. Ng, M. M. Roomi, D. Mashima, and X. Lou, "CRaaS: Cloud-based smart grid cyber range for scalable cybersecurity experiments and training," in *Proc. IEEE Int. Conf. Commun., Control, Comput. Technol. Smart Grids (SmartGridComm)*, 2024, pp. 333–339.
- [52] A. V. Jha *et al.*, "A graph-theoretic approach for modelling and resiliency analysis of synchrophasor communication networks," *Appl. Syst. Innov.*, vol. 6, no. 1, p. 7, 2023.
- [53] S. Ramírez, "FastAPI," [Online]. Available: <https://fastapi.tiangolo.com>, 2024.
- [54] G. Karypis and V. Kumar, "A parallel algorithm for multilevel graph partitioning and sparse matrix ordering," *J. Parallel Distrib. Comput.*, vol. 48, no. 1, pp. 71–95, 1998.
- [55] P. R. Amestoy, I. S. Duff, J.-Y. L'Excellent, and J. Koster, "A fully asynchronous multifrontal solver using distributed dynamic scheduling," *SIAM J. Matrix Anal. Appl.*, vol. 23, no. 1, pp. 15–41, 2001.
- [56] Message Passing Interface Forum, "MPI: A message-passing interface standard, version 4.0," 2021.
- [57] Q. Huangfu and J. A. J. Hall, "Parallelizing the dual revised simplex method," *Math. Program. Comput.*, vol. 10, no. 1, pp. 119–142, 2018.
- [58] R. D. Zimmerman, C. E. Murillo-Sánchez, and R. J. Thomas, "MATPOWER: Steady-state operations, planning, and analysis tools for power systems research and education," *IEEE Trans. Power Syst.*, vol. 26, no. 1, pp. 12–19, 2011.

Md Mazharul Islam [TODO: author biography and photo]

Sunjare Zulfiker [TODO: author biography and photo]

Masum Reza [TODO: author biography and photo]

Hafiz Abdur Rahman [TODO: author biography and photo]